Lex/Flex: Specification of Lexical Analyzers; Practical Exercises

Let assume a hypothetical lexer containing hundreds of lines of code.

> **"Would you rather manually program 40 token-recognition routines—or describe the patterns and let a tool generate the recognizer?"**

Assume the following lexer rules:

```
%%

[0-9]+                { return NUMBER; }

[a-zA-Z][a-zA-Z0-9]*    { return ID; }

"let"                 { return LET; }

[ \t\n]+              { /* ignore */ }

"+"                   { return PLUS; }

%%
```

Imagine you are working with a real programming language. The compiler team has created a lexer with hundreds of lines of rules. You don't need to understand every rule. Your task is to answer one question: What happens to `let`?

```
%%

[0-9]+                      { return NUMBER; }

[0-9]+\.[0-9]+               { return FLOAT; }


[a-zA-Z_][a-zA-Z0-9_]*       { return ID; }
```

```
"if"                    { return IF; }
"else"                  { return ELSE; }
"while"                 { return WHILE; }
"for"                   { return FOR; }
"return"                { return RETURN; }
"function"              { return FUNCTION; }
"const"                 { return CONST; }

/* ... 150+ other lexer rules ... */

"let"                   { return LET; }

/* ... many more rules ... */

"+"                     { return PLUS; }
"-"                     { return MINUS; }
"*"                     { return MULT; }
"/"                     { return DIV; }

[ \t\n]+                { /* ignore whitespace */ }

/* ... more rules ... */
%%
```

What happens when the lexer encounters `let`?

think about:

- The rule `[a-zA-Z][a-zA-Z0-9]*` recognizes `let` as an **identifier (`ID`)**.
- The rule `"let"` recognizes `let` as the **keyword (`LET`)**.
- So, **does `let` mean a variable name or a keyword?**
- What happens when two rules can recognize the same input?
- Does the **order of the rules** matter?

You see a rule for `let`. But there is also a rule that recognizes identifiers. What token will the lexer produce for `let`?

Example:

let

let x = 10

letter

Let123

1. Which rule matches `let`?
2. Which rule matches `letter`?
3. Which rule matches `let123`?
4. Does the position of the `"let"` rule matter?
5. What if the identifier rule appears **before** `"let"`?
6. What if `"let"` appears **before** the identifier rule?

Is every occurrence of `let` a keyword? What if a programmer wants to use `let` as a variable name?

[a-zA-Z_][a-zA-Z0-9_]* { return ID; }

also matches:

let

So the lexer has a **rule conflict**.

> "The lexer goes through the rules from top to bottom." **When multiple rules match the same amount of input, the rule appearing first is selected.**
>
> If a lexer has 500 rules, how can we confidently determine which rule will recognize a particular piece of input?

Opening Challenge — "Human Lex"

Let assume

let x = 25 + 10;

and the rules:

"let" → LET

[0-9]+ → NUMBER

[a-zA-Z]+ → ID

"=" → ASSIGN

"+" → PLUS

";" → SEMICOLON

whitespace → ignore

You have to manually simulate what Flex would do.

Discovery Activity

Give:

%%

[0-9]+ { return NUMBER; }

[a-zA-Z][a-zA-Z0-9]* { return ID; }

"let" { return LET; }

[\t\n]+ { /* ignore */ }

"+" { return PLUS; }

%%

"What happens to `let`?"

You may notice the potential conflict between:

ID

and:

LET

This is an excellent opportunity to discover **rule priority and longest matching**.

Concept Reveal

Explain the basic Flex model:

Regular expressions

↓

Flex specification

↓

Generated lexical analyzer

Typical structure:

definitions

%%

rules

%%

user code

Introduce:

- pattern
- action
- `yytext`
- `yyleng`
- generated scanner
- rule priority
- longest match

1. Pattern

A **pattern** describes what text the lexer should recognize.

Example:

[0-9]+

This pattern means:

> Match one or more digits.

Another:

"if"

means:

> Match exactly the keyword `if`.

A Flex rule generally looks like:

pattern action

2. Action

The **action** is the C code that Flex executes when the pattern matches.

Example:

[0-9]+ printf("NUMBER");

If the input is:

123

Flex recognizes `123` and executes:

printf("NUMBER");

A common action is:

"if" return IF;

This tells the parser:

> I found the keyword `if`; return the token `IF`.

3. `yytext`

`yytext` contains the **actual text that was matched**.

Example:

[0-9]+ printf("%s", yytext);

Input:

12345

Then:

yytext = "12345"

So:

printf("%s", yytext);

prints:

12345

Think of it as:

> **yytext = the current matched string**

4. `yyleng`

`yyleng` contains the **length of the current matched text**.

Example:

```
[0-9]+    printf("%s has length %d", yytext, yyleng);
```

For:

12345

we get:

yytext = "12345"

yyleng = 5

Think of it as:

yyleng = length of yytext

5. Generated scanner

When you write a Flex file such as:

```
%{
#include <stdio.h>
%}


%%
"if"        return IF;
[0-9]+      return NUMBER;
[a-zA-Z]+   return IDENTIFIER;
[ \t\n]+    ;
```

```
.       return yytext[0];

%%
```

Flex **generates a C program** containing the lexical scanner.

Traditionally, the generated file is:

lex.yy.c

That generated scanner contains the machinery needed to read the input, find matches, and execute your actions.

The main scanning function is:

yylex()

So conceptually:

```
Your Flex rules
      ↓
     flex
      ↓
  lex.yy.c
      ↓
   yylex()
      ↓
tokens
```

6. Rule priority

This is where your earlier question comes in.

Suppose:

```
"if"      return IF;
```

[a-zA-Z]+ return IDENTIFIER;

For input:

if

both patterns match **2 characters**.

So there is a tie.

Flex then uses **rule priority**:

> **The rule appearing earlier in the file wins.**

Therefore:

"if" return IF; /* wins */

[a-zA-Z]+ return IDENTIFIER;

`if` becomes the keyword `IF`.

7. Longest match

Before considering rule order, Flex considers **how much input each rule matches**.

This is called the **longest-match rule** or **maximal munch**.

Example:

"if" return IF;

[a-zA-Z]+ return IDENTIFIER;

Input:

ifelse

Possible matches:

"if" → "if" → 2 characters

[a-zA-Z]+ → "ifelse" → 6 characters

Flex chooses:

ifelse

because **6 > 2**.

Therefore:

ifelse → IDENTIFIER

The fact that `"if"` appears first does **not** matter here.

The most important rule to remember

Flex's decision process is:

Multiple rules match

↓

Which matches more text?

↓

LONGEST MATCH WINS

↓

If there is a tie

↓

EARLIER RULE WINS

Exam phrase to memorize:

> **Flex chooses the rule that matches the longest sequence of input. If multiple rules match the same longest sequence, the rule appearing first is selected.**

Misconception Challenge

1. "Flex itself is the compiler."

False. Flex generates a lexical analyzer.

2. "A Flex rule always matches only one character."

False.

3. "The first rule written always wins."

False in the general case; longest match is important, with rule ordering resolving certain ties.

4. "Regular expressions can describe arbitrary nested program syntax."

False.

🟢 Beginner

Create rules for:

NUMBER

ID

PLUS

MINUS

🟡 Intermediate

Add:

IF

ELSE

WHILE

==

<=

>=

🔴 Advanced

Add comments:

// this is a comment

and ensure they are ignored.

Exit Ticket

1. What is the purpose of Flex?
2. What does `yytext` represent?
3. **Why can `ifelse` become an identifier even though `if` is a keyword?**

Transition

> "Our lexer can now tell us that the input contains `LET`, `ID`, `ASSIGN`, and `NUMBER`. But can it tell whether those tokens are arranged into a valid program?"

You will discover the need for **syntax analysis**.

Lex and **Flex** are tools used to create a **lexical analyzer (lexer)** for a compiler.

Think of them as tools that take your **patterns/rules** and automatically generate C code that recognizes tokens.

Simple example

Suppose you want to recognize numbers and words:

```
%%

[0-9]+       { printf("NUMBER: %s\n", yytext); }

[a-zA-Z]+    { printf("WORD: %s\n", yytext); }

[ \t\n]      { /* ignore whitespace */ }

%%
```

If the input is:

hello 123 world

The lexer identifies:

WORD: hello

NUMBER: 123

WORD: world

Lex vs Flex

Lex	Flex
Original Unix lexer generator	Modern/free alternative to Lex
Older	More commonly used today
Generates lexer code	Generates lexer code

Uses rules like `[0-9]+`	Uses essentially the same syntax

Flex = Fast Lexical Analyzer Generator.

`win_flex_bison-2.5.25.zip`

nside the ZIP you'll get roughly:

win_flex.exe → Lexical analyzer
win_bison.exe → Parser generator

What each one does

`win_flex.exe`

- Reads your `.l` file
- Generates a lexer
- Recognizes tokens
- Gives you things like `yytext` and `yyleng`

`win_bison.exe`

- Reads your `.y` file
- Generates a parser
- Checks whether the token sequence follows your grammar
- Can **build a parse tree/AST** if you write the tree-building code

So:

win_flex
  ↓
Tokens
  ↓
win_bison
  ↓
Parser
  ↓
Parse Tree / AST